

Design Document

for

Caregiver Android Team

by

Thomas Cham, Rohit Jayaram, Tobi Bakare, Basil Khan,
Nicholas Robbins, Leonid Mezhibovskii

**Senior Design
Project Advisor:**

Mr. Jeff Salvage

Stakeholder:

Mrs. Karen Velocci

Table of Contents

- 1. Document History
- 2. Introduction
 - 2.1 Purpose
 - 2.2 Scope
 - 2.3 Design Goals
 - 2.4 Definitions
 - 2.5 References
- 3. System Overview
 - 3.1 Context Diagram
 - 3.2 Technologies Used
 - 3.3 Architecture
 - 3.4 Sequence Diagram
- 4. Detailed System Architecture
- 5. Data Model

1. Document History

Editors	Date	Reason	Version
Thomas Cham, Rohit Jayaram, Tobi Bakare, Basil Khan, Nicholas Robbins, Leonid Mezhibovskii	2/7/2022	Initial Draft/Skeleton of Design Doc	1.0
Thomas Cham, Rohit Jayaram, Tobi Bakare, Basil Khan, Nicholas Robbins, Leonid Mezhibovskii	2/14/2022	Updating document	2.0
Thomas Cham, Rohit Jayaram, Tobi Bakare, Basil Khan, Nicholas Robbins, Leonid Mezhibovskii	3/4/2022	Updating document	3.0

2. Introduction

2.1 Purpose

The purpose of this document is to describe the system architecture and design specifications for the Caregiver app. The design decisions described in this document are in line with the requirements specified in the Caregiver SRS.

2.2 Scope

The scope of this document is to cover software architecture for the initial release of Caregiver. The intended audience of this document is designers, developers, and testers of Caregiver.

2.3 Design Goals

- **Fit for Purpose** - This application achieves its useful functions without any unnecessary components added to cause confusion for any user. Understanding the requirements of both the caregiver and caregivee is essential for the facility.
- **Convenience** - One of the main purposes of this application is making things easier for the users. The application passes a task or activity in a better, easier or simpler way from the caregivers to the caregivees. With a minimum number of steps or physical/emotional strength, the user is able to complete the tasks and get rewarded for them with a picture or a video that they like.

2.4 Definitions

AWS - provides on-demand cloud computing platforms and APIs to individuals, companies, and governments

AWS S3 - a simple Storage Service is a service offered by Amazon Web Services that provides object storage through a web service interface.

Axios - A Javascript library used to make HTTP requests from node.js or XMLHttpRequests from the browser.

Architecture Diagram - a visual representation of all the elements that make up part, or all, of a system.

Caregiver & Caregivee - classification for the two main users. The caregiver can be a family member or paid helper while the caregivee is the person the caregiver is responsible for.

Context Diagram- shows the interactions between a system and other actors(external factors) with which the system is designed to interface.

Expo- is an open-source platform for making universal native apps for Android, iOS, and the web with JavaScript and React.

Flask Server - REST API for Python with support for multithreading and performant networking.

Hypertext Transfer Protocol (HTTP) - An application-layer protocol for transmitting hypermedia documents, such as HTML.

PostgreSQL- also known as Postgres, is a free and open-source relational database management system emphasizing extensibility and SQL compliance.

Python- A high-level general-purpose programming language used for data analytics, machine learning, and even design.

React Native- used to develop applications for Android, Android TV, iOS, macOS, tvOS, Web, Windows and UWP

Sequence Diagram- depicts interaction between objects in a sequential order

Structured Query Language (SQL) - a standardized programming language that is used to manage relational databases and perform various operations on the data in them.

XMLHttpRequests- Are objects used to interact with servers

3. System Overview

3.1 Context Diagram

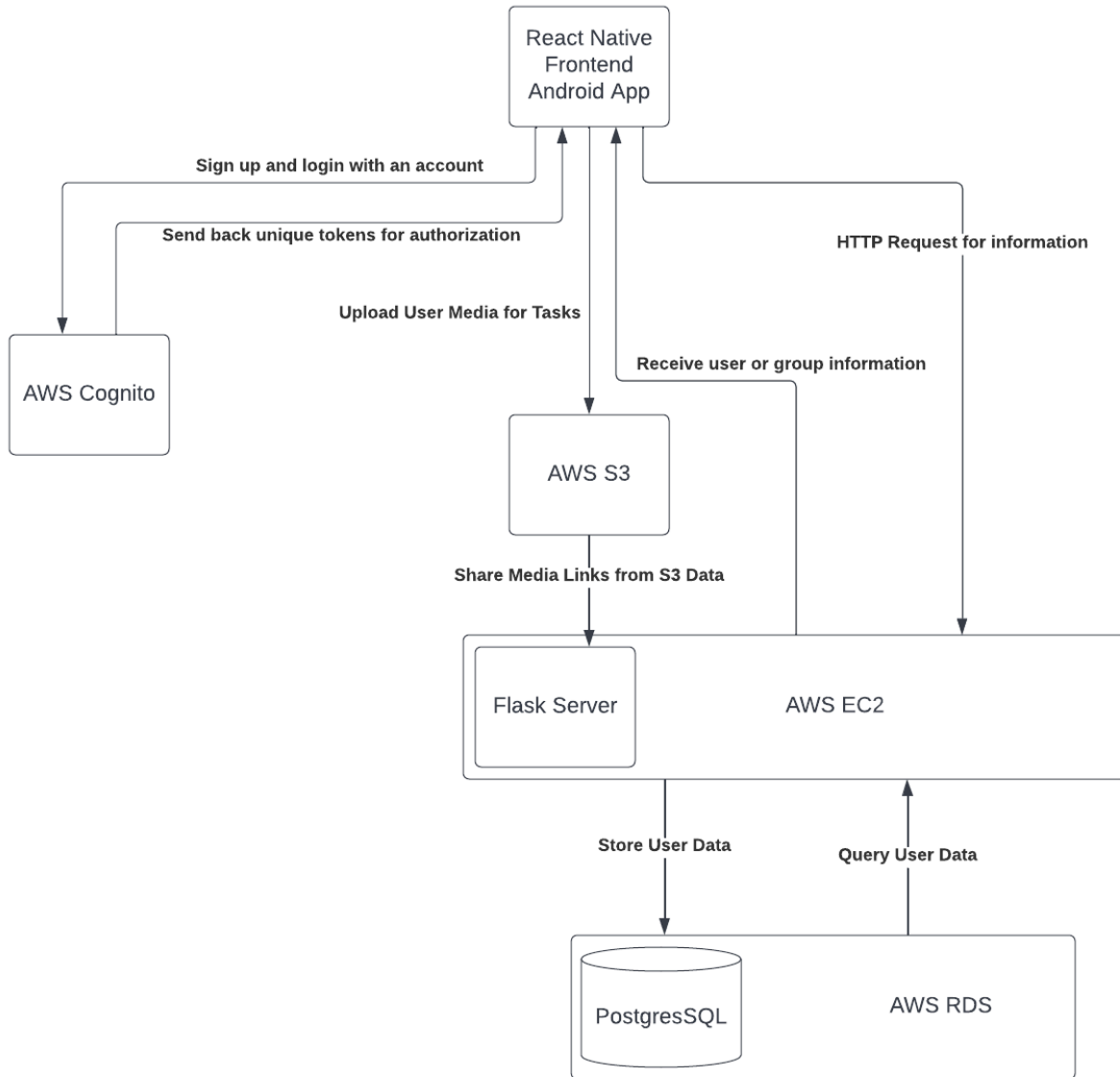


Figure 1. Caregiver Context Diagram

Figure 1 shows the interactions between the three primary components of the system: The React Native App, the Flask Server, and the PostgreSQL Database. The user interacts with the front-end of the mobile app, which then can send an HTTP request to the Flask Server using Axios. Upon receiving a request, the server will then query the PostgreSQL database, which then returns the appropriate data model. When the user requests to load data, such as tasks or notifications, a GET request is made to the server, which queries the database and returns the

appropriate model. When the user saves or creates data in the app, a POST or PUT request is made to the server, which updates or inserts the model in the database. The React Native UI also uploads media to AWS S3 which is then shared by the flask server when another user in the same group requests a particular piece of media.

3.2 Technologies Used

Caregiver uses React Native and Expo for the front end of the mobile app, which utilizes javascript, html, and css. It uses Axios to send HTTP requests to the server, and Flask and Python for the server itself. The Flask server communicates with a PostgreSQL database, which stores models for tasks, notifications, and caregiver/caregiver profiles. The app also uses AWS S3 buckets to store images, videos, and audio that is uploaded from the app.

3.3 Architecture Diagram

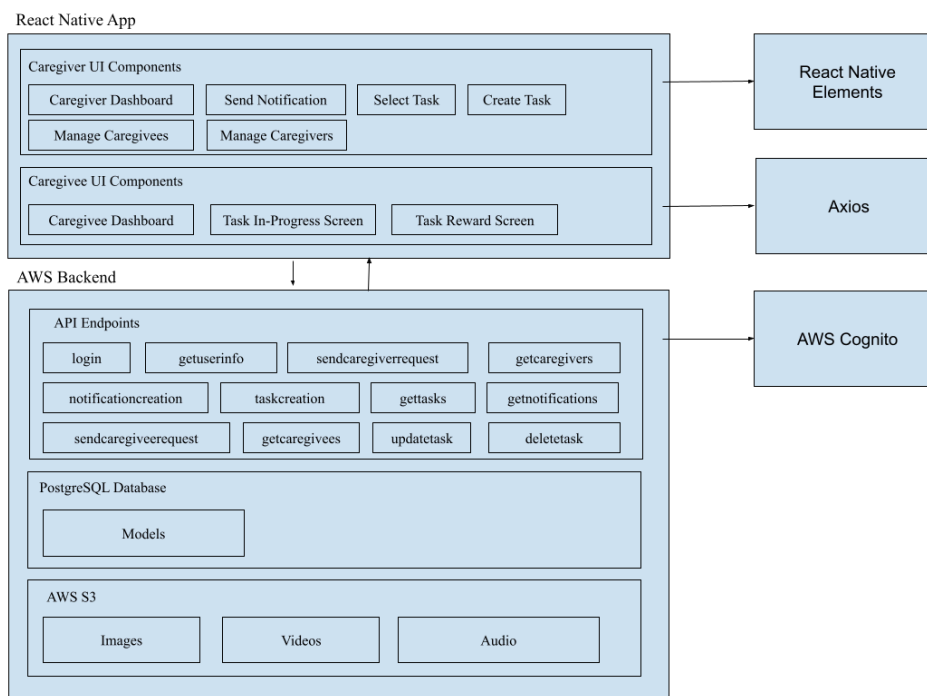


Figure 2. Architecture Diagram

Figure 2 displays the architecture for Caregiver. The diagram shows the components of both the React Native frontend and the AWS backend. The frontend of the app is split into both the caregiver side of the app, which is used by caregivers, and the caregiver side of the app, which is used by caregivers. Both of these sides contain their own React Native components which the user can interact with from the app. The frontend utilizes external libraries such as React Native Elements, and Axios for making HTTP requests. On the server side, there is a list of API endpoints that can be called from the front-end using an Axios HTTP request. These are broken into GET requests for requesting data, as well as POST and PUT requests for updating and

inserting data. The server interacts with the PostgreSQL database by retrieving, saving, or inserting the appropriate model when an endpoint is called. The frontend also interacts with AWS S3 buckets for storing images, videos, and audio for tasks.

3.4 Sequence Diagram

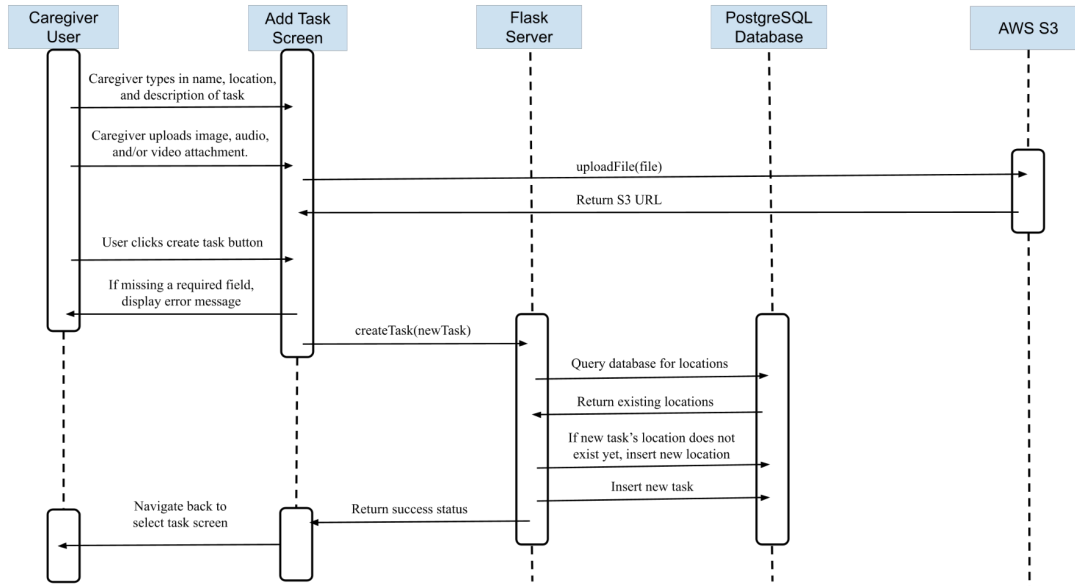


Figure 3. Sequence diagram for creating a new task

Figure 3 shows the process for creating a new task as a caregiver in the app. After navigating to the Add Task Screen, the caregiver types in the name, location, and description for the new task. The caregiver then uploads any media attachments, such as images, videos, or audio attachments. The Add Task Screen then uploads the media files to AWS S3, which returns the URL for the asset. When the user clicks the create task button, if they are missing a required field, the UI will display an error message, otherwise it makes an HTTP request to the **taskcreation** endpoint on the server. When the server receives the request, it queries the database for locations, and checks if the new task's location exists yet. If the location does not exist, the server inserts the new location into the database. Finally, the server inserts the new task and returns a success status back to the UI. The UI then navigates the user back to the select task screen.

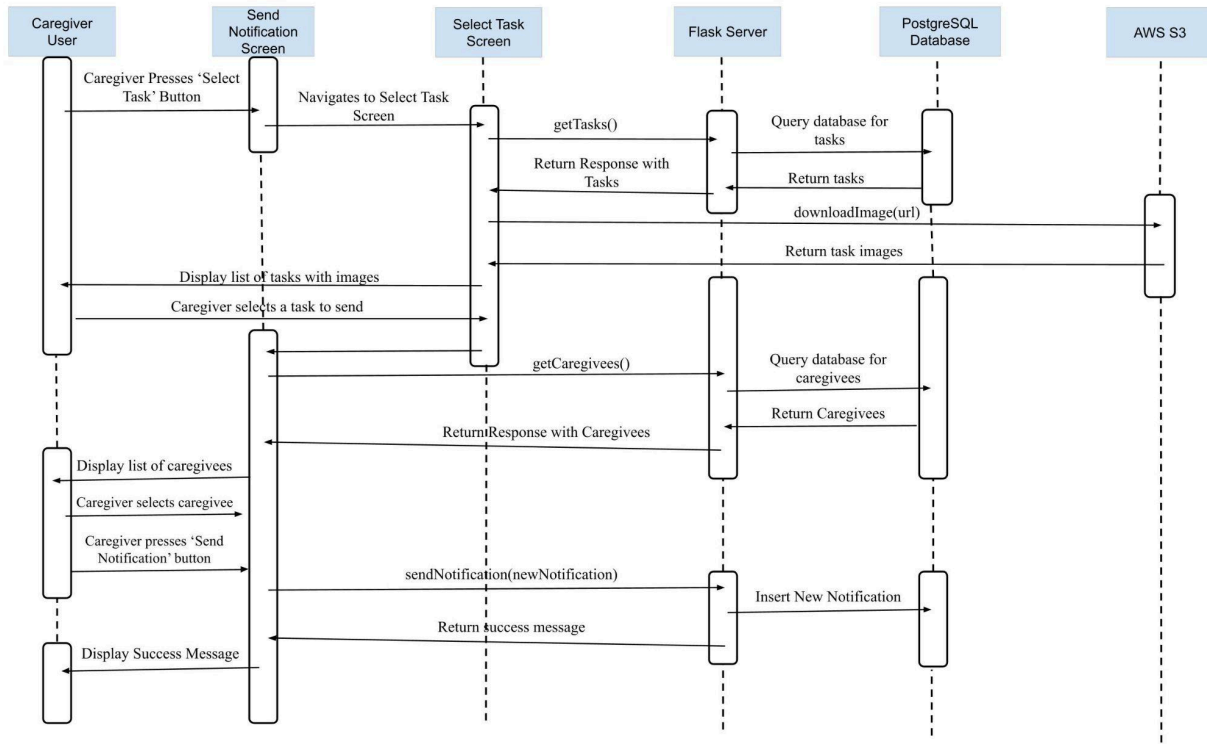


Figure 4. Sequence diagram for sending a notification

Figure 4 shows the process for creating and sending a notification as a caregiver in the app. After navigating to the **Send Notification** screen, the caregiver presses the ‘Select Task’ button, which navigates them to the **Select Task** screen. The select task screen then makes an HTTP request to the **gettasks** endpoint on the server. When the server receives the request, it queries the database for the user’s created tasks and returns them back to the select task screen, which displays the tasks to the user along with their images that are downloaded from S3. The caregiver then selects a task from the list on the screen, and is navigated back to the **Send Notification** screen. The send notification screen makes an HTTP request to the **getcargives** endpoint on the server. Once the server receives this request, it queries the database for caregivers and returns those back to the screen. The screen displays a list of caregivers to the user to select from. The caregiver then selects a caregiver to send the notification to, and then presses the ‘Send Notification’ button. The screen then makes an HTTP request to the **notificationcreation** endpoint on the server, which inserts the new notification to the database and returns a success message. Finally, the screen displays a success message to the user to confirm that the notification has been sent.

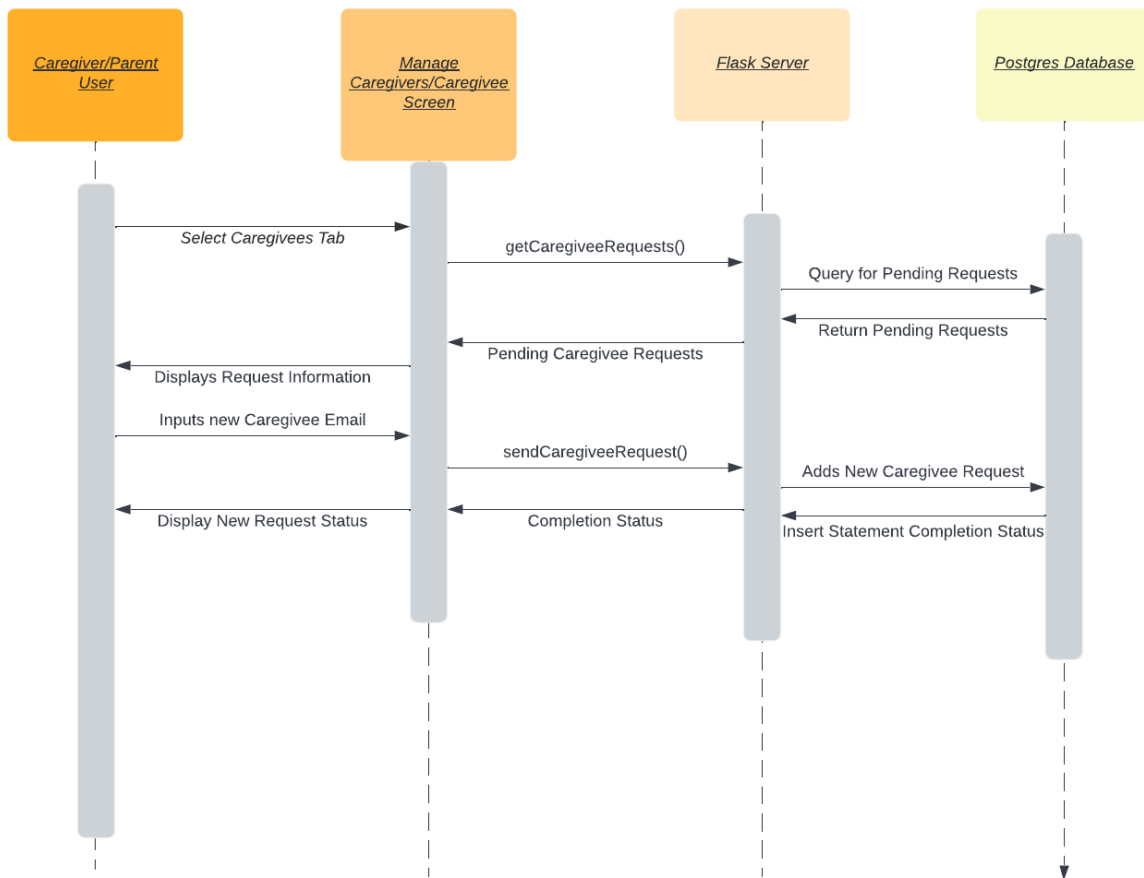


Figure 5. Sequence Diagram for Sending a Caregiver Request.

Figure 5 shows the process for sending a caregiver request from the perspective of a caregiver or parent. The parent first navigates to the managed caregivers/caregiver page where he or she receives the current pending requests. Then the caregiver or parent will input the caregiver email that they wish to add to their own group. This will send a request to the backend Flask server that will check to see the validity of the request. If the requested email is a valid caregiver the server will insert a new request into the Postgres database and return a success response. This will propagate through the management screen and finally display the status of the pending request.